

1.1-Ingeniería de Software y sus disciplinas

Esp. Ing. Mónica Colombo

Ciclo Lectivo: 2026



One principal problem of educating software engineers is that they will not use a new method until they believe it works and, more importantly, that they will not believe the method will work until they see it for themselves

Watts S. Mumphrey

La incapacidad de las organizaciones y de los profesionales para predecir tiempo, esfuerzos y costos en el desarrollo del software producidos en un proyecto, son dos de las principales bases sobre las que surge la Ingeniería del Software como una disciplina científica.



Perspectivas históricas del Sw

■ Década 1950-60:

- "Software como un añadido"
- Aplicaciones sencillas
- Desarrollo artesanal, a medida
- Lenguajes de bajo nivel

■ Década 1960-70:

- Software como producto
- Primeras aplicaciones complejas
- Década lenguajes y compilación
- "Crisis del software"

■ Década 1970-80:

- Programación estructurada
- Modelo relacional
- Primeras etapas Ingeniería del Software
- Primeros métodos estructurados
- Modelado de datos

■ Década 1980-90:

- Programación OO
- 4GLs
- C/S
- Tecnología de SGBDs, SOs
- Métodos estructurados
- Primeros métodos OO
- Tecnología CASE (1ª generación)

■ Década 1990-00

- Generalización POO
- Programación visual
- Tecnología de componentes
- Interoperabilidad (CORBA)
- Nuevas plataformas (Java, .NET)
- Análisis/Diseño OO
 - Fin "Guerra de los métodos"
 - UML (Unified Modeling Language, 1997)
- Patrones
- Tecnología CASE (2ª generación)
- Popularización de Internet

■ Década 2000-10

- Generalización comercio electrónico
- Web 2.0
- Desarrollo web
- Seguridad
- Arquitecturas basadas en servicios (SOA)
- Métodos ágiles
- GSD: Global Software Development
- Desarrollo open source
- MDE: Model-Driven Engineering

AUMENTAN los problemas del desarrollo de software:

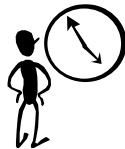
- ➔ Subexplotación del potencial del hardware
- ➔ Incapacidad de atender a la demanda
- ➔ Incapacidad de mantener el software existente

Naturaleza y problemas del desarrollo de software

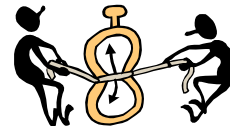
- El software como elemento lógico.
 - **Se desarrolla, no se fabrica:**
 - Calidad del diseño.
 - Costes más importantes en la ingeniería
 - Gestión especial de los proyectos
 - Se “deteriora” con el mantenimiento
 - Desarrollo a medida (ausencia de componentes)
- La **“crisis” del software**: *problemas que aparecen en el desarrollo del software al desarrollar, mantener y atender la demanda de nuevas aplicaciones.*



Planificación y estimaciones imprecisas



Sin tiempo para recoger datos históricos



Dificultad de mantener el software existente



Insatisfacción del cliente



Calidad



Baja productividad

Contenidos: 1.1-Software e Ingeniería de Software

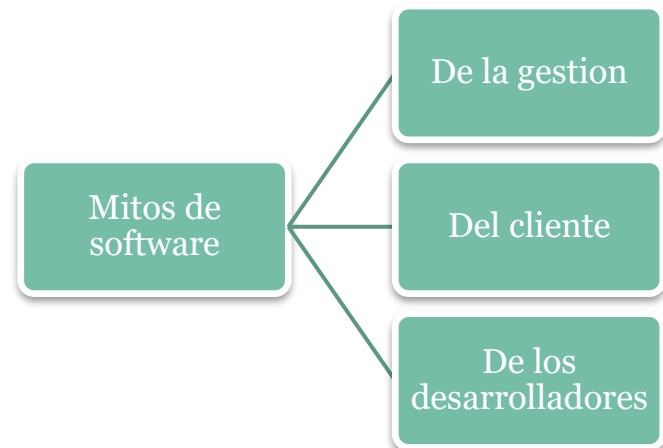
1. Mitos del software.
2. Características y aplicaciones del software.
3. Definición y diferencias de ingeniería de software.
4. Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos y Procesos de desarrollo de Software
5. Conceptos de Ingeniería de software y requerimientos
6. Métodos de ingeniería de software.
7. Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.
8. Retos de la ingeniería de software.
9. Usabilidad en el desarrollo de Software
10. Guia SWEBOK

BIBLIOGRAFIA

1. "INGENIERIA DE SOFTWARE" 6TA. ED. IAN SOMMERVILLE.- ED. ADISON WESLEY (2002)
2. "INGENIERIA DEL SOFTWARE: UN ENFOQUE PRACTICO" 7MA ED.DE ROGER S.PRESSMAN ED. MCGRAWHILL (2010)
3. "FABRICAS DE SOFTWARE: EXPERIENCIAS, TECNOLOGIAS Y ORGANIZACIÓN" DE MARIO PIATTINI Y JAVIER GARZAS PARRA- ED ALFAOMEGA RA-MA (2007)
4. "CALIDAD EN EL DESARROLLO Y MANTENIMIENTO DEL SOFTWARE" DE MARIO PIATTINI Y FELIX GARCIA- ED ALFAOMEGA RA-MA (2003)
5. "INGENIERIA DE SOFTWARE, UNA PERSPECTIVA ORIENTADA A OBJETOS" DE ERIC BRAUDE, ED ALFAOMEGA (2005)
6. "INGENIERIA DEL SOFTWARE, UN ENFOQUE DESDE LA GUIA SWEBOK" DE SALVADOR SANCHEZ, MIGUEL SICILIA Y DANIEL RODRIGUEZ- ED ALFAOMEGA (2011)

1-Mitos del software

- Los mitos del software -creencias acerca del software y de los procesos empleados para construirlo- se pueden rastrear hasta los primeros días de la computación.
- Estos mitos propagaron información errónea y confusión.
- Los mitos tienen ciertos atributos que los convierten en insidiosos.



Mitos del software: Mitos de la Gestión

- Los gestores con responsabilidad sobre el software, como los gestores en la mayoría de las disciplinas, están normalmente bajo la presión de cumplir las propuestas, hacer que no se retrase el proyecto y mejorar la calidad. Un gestor de software se agarra frecuentemente a un mito del software.

MITO 1

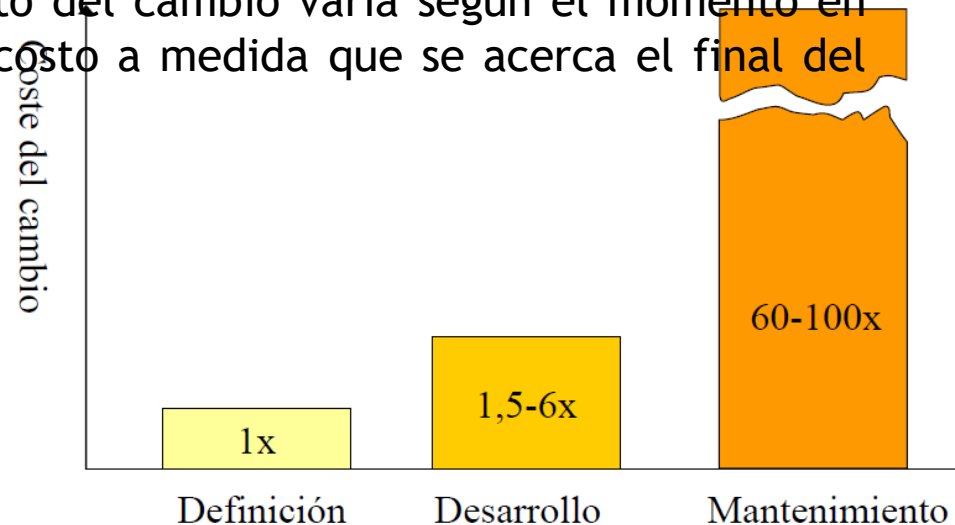
- Tenemos escritos estándares y procedimientos para construir sw, no le da a la gente todo lo que necesita?
 - Si no hay estándares se pasa a depender de costumbres
- Mi gente dispone de las herramientas de desarrollo de sw más avanzadas, después de todo les compramos el hw mas moderno
 - Se necesita algo mas que hw para hacer desarrollo de sw de calidad.
- Si se falla en la planificación, se puede añadir mas programadores y adelantar el tiempo perdido.
 - Puede añadirse gente pero solo de manera planificada y bien coordinada. Cuando se añaden nuevos RRHH, necesitan tiempo para aprender y comunicarse, reduciendo el tiempo productivo

Mitos del software: Mitos del Cliente

- En muchos casos, el cliente cree en los mitos que existen sobre el software, debido a que los “gestores y desarrolladores de software hacen muy poco para corregir la mala información”.
- Los mitos conducen a que el cliente se cree una falsa expectativa y, finalmente, quede insatisfecho con el software desarrollado.

MITO 2

- Si los requisitos del proyecto cambian continuamente, los cambios pueden acomodarse fácilmente, ya que el software es flexible.
 - Los requisitos cambian pero el impacto del cambio varia según el momento en que se introduzca, creciendo mas el costo a medida que se acerca el final del proyecto.



Mitos del software: Mitos de los desarrolladores

- Los mitos en los que aun creen muchos desarrolladores se han ido fomentando durante 50 años de cultura informática. Durante los primeros días del desarrollo del software, la programación se veía como un arte. Las viejas formas y actitudes tardan en morir.

MITO 3

- **Una vez que escribimos el programa y hacemos que funcione, nuestro trabajo ha terminado.**
 - Entre el 60 y 80% de todo el esfuerzo dedicado al programa se realiza una vez que se haya entregado por primera vez para ser testeado
- **Hasta que no tengo el programa, ejecutándose, realmente no tengo forma de probar su calidad.**
 - Desde el principio del proyecto se pueden aplicar revisiones técnicas formales, revisiones por pares y testeos unitarios, entre otros métodos.
- **Lo único que se entrega al terminar el proyecto es el programa funcionando**
 - Un programa que funciona es solo una parte de una configuración de software que incluye muchos elementos. La documentación proporciona el fundamento para un buen desarrollo y da guías para el mantenimiento del sw (especificación de requisitos, plan del proyecto, arquitectura, diseño detallado, DER, wireframes, prototipos, repositorios de código y documentación, etc.)

2-Características y aplicaciones del software

CARACTERISTICAS

1. El Sw se desarrolla, no se fabrica en un sentido clásico

- El enfoque de desarrollo de Hw y de Sw se adquiere la calidad mediante **un buen diseño**
- Ambas actividades :
 - dependen de personas, pero la relación entre las personas dedicadas y el trabajo realizado es diferente.
 - requieren la construcción de un producto pero los enfoques son diferentes
- **Los costos del Sw se encuentran en la ingeniería, por lo cual los proyectos de Sw no se pueden encarar como si fueran de fabricación.**

2. El Sw no se estropea

- El Hw exhibe relativamente muchos fallos al principio de su vida por defectos de fabricación, luego se reduce y cuando empieza a desgastarse, la tasa de fallos se incrementa. Si una pieza se rompe, esta se reemplaza
- El Sw no es susceptible a los males del entorno que hacen que el Hw se estropee. **A medida que se deteriora se introducen nuevos defectos y aparecen picos por que se deteriora el Sw debido a los cambios. Si un componente de Sw tiene errores en el diseño o procesos no se puede reemplazar. El mantenimiento del Sw tiene una complejidad considerablemente mayor que la del mantenimiento del Hw.**

Características y aplicaciones del software

CARACTERISTICAS

- La mayoría del software se construye a medida, en vez de ensamblar componentes existentes.
 - No existen catálogos de componentes de software. Se puede comprar software ya desarrollado, pero solo como una unidad completa, no como componentes que pueden re-ensamblarse en nuevos programas.
 - La **reutilización** es una característica que debe implementarse para que pueda volver a ser reutilizado en muchos programas diferentes, ya sea algoritmos o estructuras de datos.
 - Los componentes reutilizables encapsulan tanto datos como procesos que se aplican a los datos, como por ejemplo interfaces graficas de usuario.

3-Definición y diferencias de ingeniería de software

- El término ingeniería del software empezó a usarse a finales de la **década de los 60**, para expresar el área de conocimiento que se estaba desarrollando en torno a las problemáticas que ofrecía el software en ese momento.
- En esa época, el crecimiento espectacular de la demanda de sistemas de computación cada vez más y más complejos, asociado a la inmadurez del propio sector informático (totalmente ligado al electrónico) y a la falta de métodos y recursos, provocó lo que se llamó la **crisis del software** (en palabras de Edsger Dijkstra) entre los años **1965 y 1985**.
- Desde **1985 hasta el presente**, han ido apareciendo **herramientas, metodologías y tecnologías** que se presentaban como la solución definitiva al problema de la planificación, previsión de costes y aseguramiento de la calidad en el desarrollo de software.

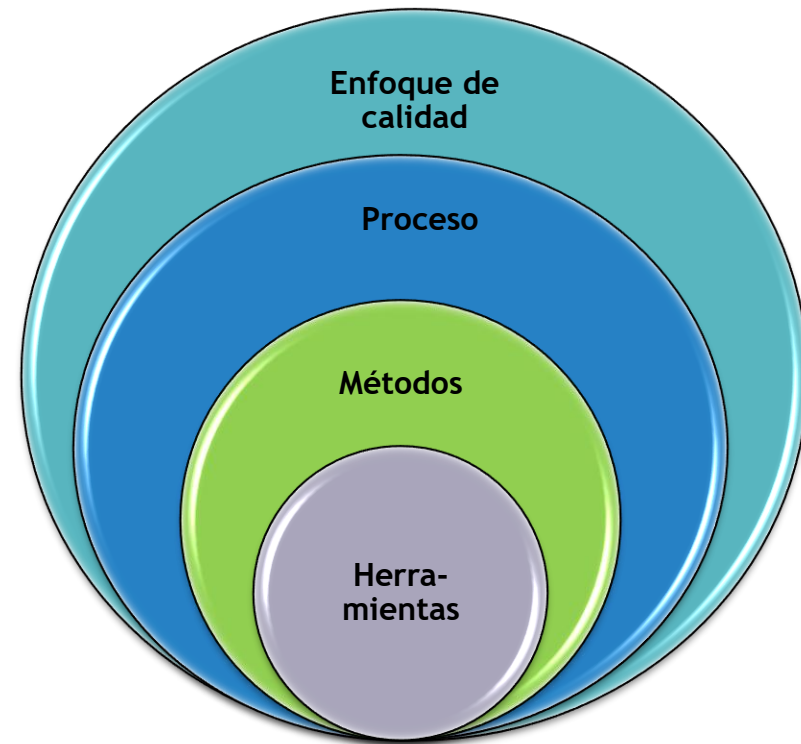
Definición: Ingeniería de Software

Es una disciplina que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema, hasta el mantenimiento de este después de que se utiliza.

- La ingeniería de software trata de **actividades llevadas a cabo por personas**(ingenieros, usuarios y otros intervinientes) **que producen, usan o modifican artefactos, métodos, especificaciones y modelos, entre otros.**
- Las nuevas metodologías suponen un enfoque integral del problema, **abarcando todas las fases**, que en su mayoría no se consideraban en los desarrollos tradicionales.
- Estas tecnologías constituyen la denominada "Ingeniería del Software", que se puede definir como **"el tratamiento sistemático de todas las fases del ciclo de vida del software"**.

Ingeniería de Sw como tecnología multicapa

- Pressman caracteriza la Ingeniería de Software como **“una tecnología multicapa”**
- Cualquier disciplina de ingeniería (incluida la ingeniería del software) debe descansar sobre un **esfuerzo de organización de calidad**.
- La gestión total de la calidad y las filosofías similares fomentan una cultura continua de mejoras de procesos que conduce al desarrollo de enfoques cada vez más robustos para la ingeniería del software.



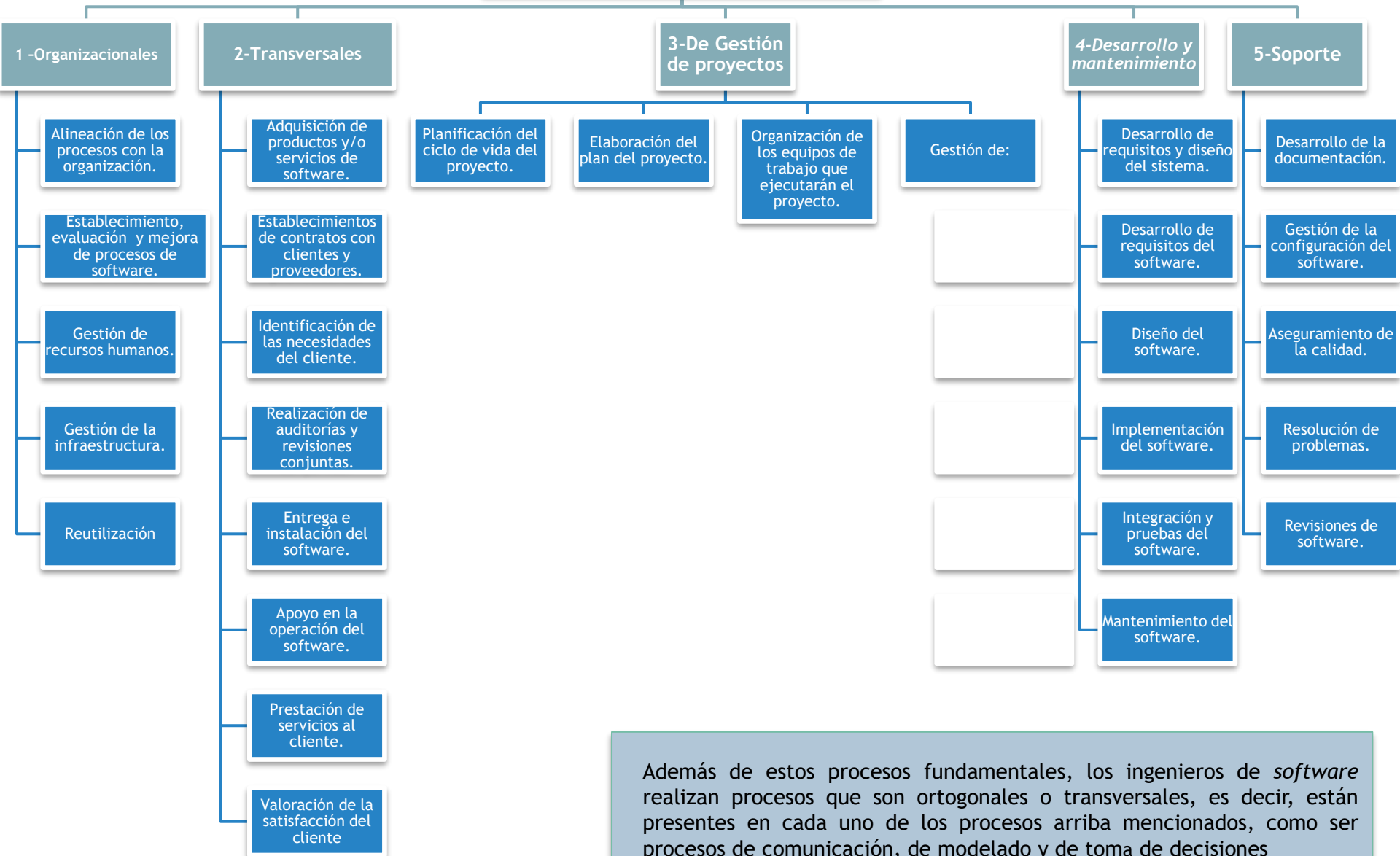
Ingeniería de Sw como tecnología multicapa

- El fundamento de la ingeniería de software es **la capa del proceso**.
 - El proceso define un **marco de trabajo** para un conjunto de áreas clave, las cuales forman la base del control de gestión de proyectos de software y establecen el contexto en el cual: se aplican los métodos técnicos, se producen resultados de trabajo, se establecen hitos, se asegura la calidad y el cambio se gestiona adecuadamente.

Dado lo anterior, el objetivo de la ingeniería de software es lograr productos de software de calidad (tanto en su forma final como durante su elaboración), mediante un proceso apoyado por **métodos** y **herramientas**.

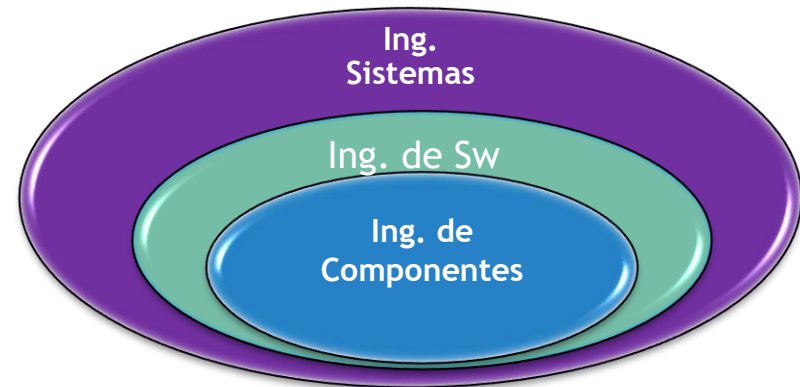
- Los **métodos** de la ingeniería de software indican **cómo construir técnicamente el software**.
 - Los métodos abarcan una gran gama de tareas que incluyen análisis de requisitos, diseño, construcción de programas, pruebas y mantenimiento.
 - Estos métodos dependen de un conjunto de principios básicos que gobiernan cada área de la tecnología e incluyen actividades de modelado y otras técnicas descriptivas.

Procesos de Ingeniería de Software



Diferencias

- La **Ingeniería en Sistemas** se involucran en las especificaciones del sistema, diseño, definición de su arquitectura, desarrollo , implementación y en la integración para crear el sistema de información final. Se enfoca en la gestión de la información.
- La **Ingeniería de Software** se centra en el desarrollo de software, desde la definición de los requerimientos hasta su mantenimiento.
 - La Ing.Sw es parte del proceso.
 - Están mas relacionados con la ingeniería de componentes.
 - Es mas joven que la Ing. en Sistemas
 - Utiliza procesos de Ing. en Sistemas, como por ejemplo el modelado de casos de uso, gestión de configuración, etc.,
 - La Ingeniería de Software **utiliza la Ingeniería de Componentes para crear software modular y reutilizable**.
- La **Ingeniería de Componentes** es una metodología dentro de la Ingeniería de Software que se basa en la creación y reutilización de componentes de software independientes.



4-Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos y Procesos de desarrollo de Software

Proceso

- conjunto ordenado de tareas, una serie de pasos que involucran actividades, restricciones y recursos, que producen una salida determinada

Proceso de software

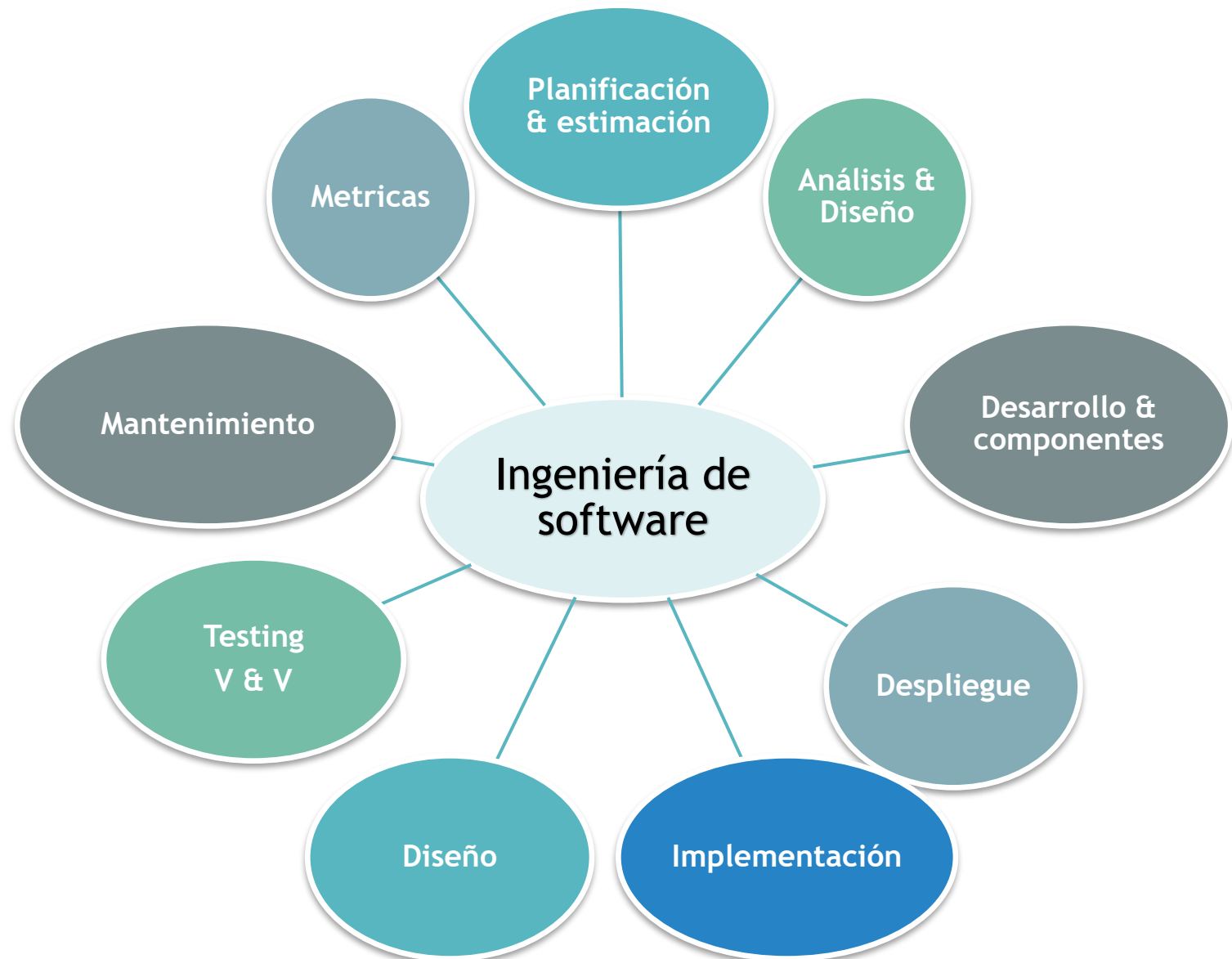
- conjunto de actividades necesarias para transformar los requisitos de un usuario en un sistema software



Características

- tiene una serie de **actividades principales**
- **utiliza recursos**, está sujeto a restricciones y genera productos intermedios y finales
- compuesto por **subprocesos** que se encadenan de alguna forma
- cada actividad tiene sus criterios de **entrada y salida**, que permiten conocer cuando comienza y termina dicha actividad
- existen principios orientadores que explican las **metas de cada actividad**

Ingeniería de Sw



Ciclos de vida y su influencia en la Administración de Proyectos y Procesos de desarrollo de Software

- Cuando un proceso implica la construcción de un producto, se suele llamar **ciclo de vida**
 - aportan consistencia y estructura sobre el conjunto de actividades, lo que permite realizar la misma tarea correctamente de forma repetida
 - existen diferentes modelos de proceso
- El software hace referencia a los programas y toda la información asociada y materiales necesarios para soportar su instalación, operación, reparación y mejora.

Para construir un nuevo elemento software se necesita:

- Detallar las especificaciones
 - Diseñar la solución
 - Codificar el algoritmo
 - Probar el programa
 - Documentar
 - Mantener
- ➡ Es lo que se conoce **como el ciclo de vida del software.**

Modelos de Proceso

- **Modelos de proceso genéricos:**

1. Modelo en Cascada (Waterfall)
2. Modelo en espiral
3. Desarrollo Evolutivo
4. Desarrollo Incremental
5. Transformaciones Formales
6. Integración a partir de componentes reutilizables.

- **Prototipado**

- Un modelo sirve de prototipo para la construcción del sistema final.

- **Transformación Formal**

- Un modelo matemático del sistema se transforma formalmente en los requerimientos en un diseño para luego programarlos e implementarlos.

- **Desarrollo basado en Componentes reutilizables**

- El sistema es ensamblado a partir de componentes existentes.

Fases genéricas en el ciclo de vida del SW



Tareas:

- Ingeniería de sistemas
- Planificación del proyecto del SW
- Análisis

Tareas:

- Diseño del SW
- Prototipado
- Generación de código
- Prueba

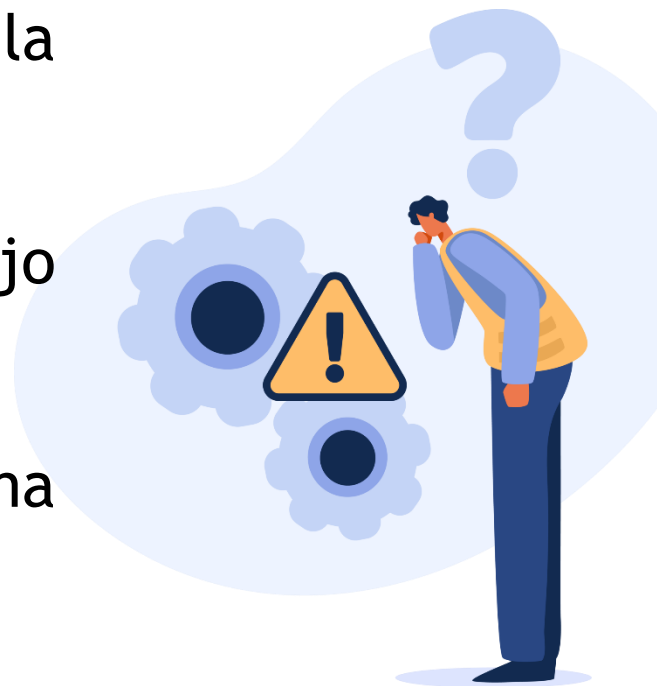
Cambios:

- Corrección
- Adaptación
- Mejora
- Prevención

5-Conceptos de Ingeniería de software y requerimientos

Problemas

- Los usuarios no saben lo que quieren.
- Un sistema tiene muchos usuarios y ninguno tiene una visión de conjunto.
- No saben cómo hacer más eficiente la operación en su conjunto
- No saben qué partes de su trabajo pueden transformarse en software.
- No saben detallar lo que saben de forma precisa.



Leyes & Lemas

Hay 3 Leyes que nos recuerdan los problemas de la prescripción, de intentar adivinar el futuro, y tenerlo muy estudiado y documentado, antes de empezar a crear:

Ley de Ziv

- “Los requisitos nunca se entienden completamente”

Ley de Humphrey

- “Los usuarios no saben realmente el software que quieren hasta que lo ven funcionando”

Lema de Wegner

- “Un sistema interactivo nunca puede ser ni especificado ni testeado por completo”

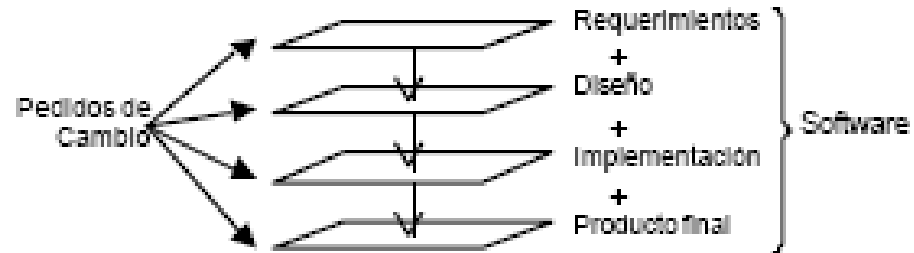
Comprensión de los requerimientos

Requerimientos: Son las **capacidades** que debe tener un sistema para la solución de un problema.

- Deben incluir todos los aspectos necesarios para **satisfacer las necesidades de los clientes**, entre las que se incluyen criterios de calidad.
- Las descripciones de los servicios y restricciones son los **requerimientos** de los sistemas

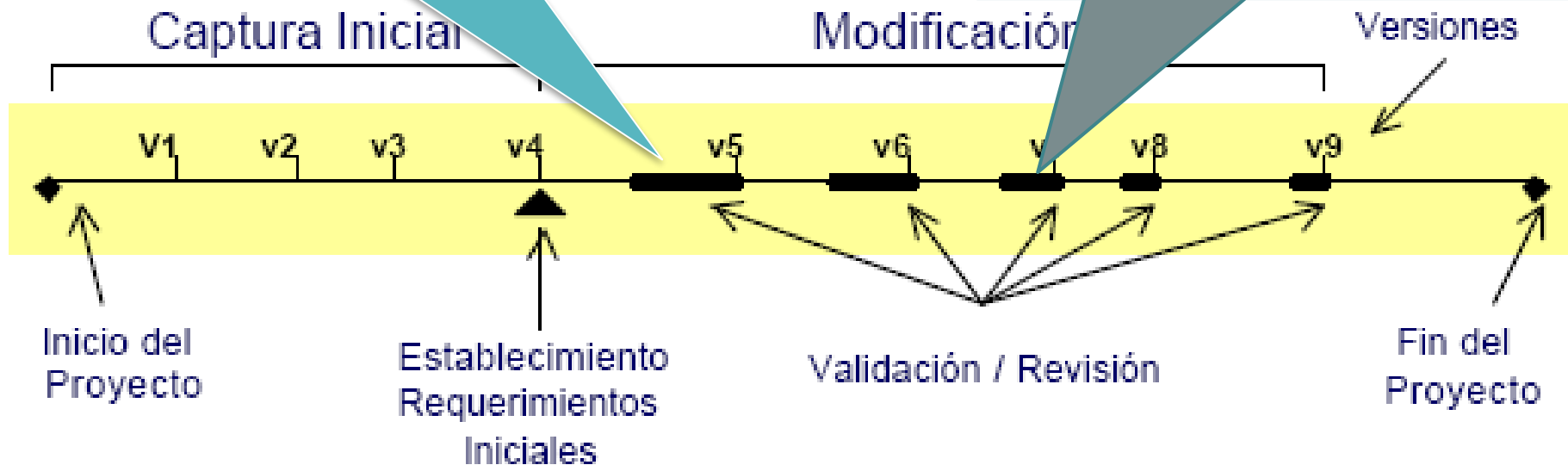
Ciclo de Vida de los Requerimientos

1. Captura Inicial
2. Modificaciones



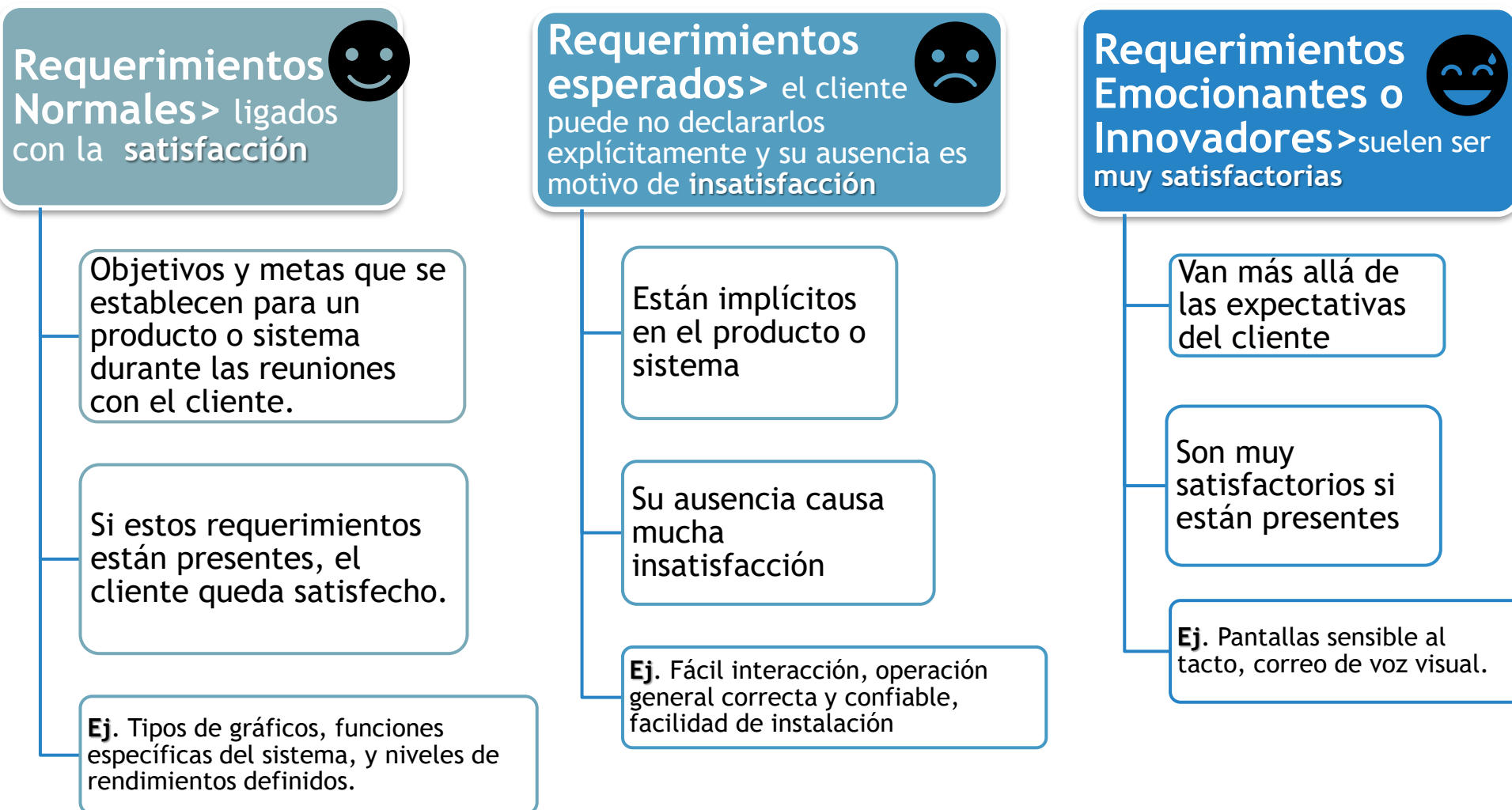
El contexto del negocio cambió y es necesario modificar requerimientos para adecuar el software al nuevo contexto

Al avanzar en el desarrollo se detectó que un requerimiento no era correcto o válido, o que no estaba en el nivel de detalle necesario



Indagación de requerimientos

- Despliegue de la función de calidad - **DFC**: se identifican 3 tipos de requerimientos:



Indagación de requerimientos: PASOS

Requerimientos Definición/Especificación

Definición de Requerimientos

- Una declaración en un Lenguaje Natural incluye los diagramas de los servicios del sistema y sus límites operacionales.
- Escrito para clientes.

Especificación de Requerimientos

- Un documento estructurado con descripción o detalle de los servicios del sistema.
- Escrito como un contrato entre el cliente y el contratista.

Especificación de Software

- Descripción detallada de software, la cual, puede servir como una base para diseño o implementación.
- Escrito para el equipo de desarrollo del sistema

En qué se diferencian los “requisitos de usuario” DRU de los “requisitos del software” ERS?

El **DRU o Requisitos del usuario**

- se escribe desde el punto de vista del usuario/cliente/interesado.
- no poseen demasiado nivel de detalle, son de más alto nivel
- se incluye la descripción del problema actual (razones por las que el sistema de trabajo actual es insatisfactorio) y las metas que se espera lograr con la construcción del nuevo sistema.

La **ERS o Requisitos del Sw**

- desarrolla mucho más los contenidos de la DRU.
- los requisitos del software son más detallados y se escriben para el equipo de desarrollo.
- Contiene la respuesta a la pregunta ¿Qué características debe poseer un sistema que nos permita alcanzar los objetivos, y evitar los problemas, expuestos en la DRU?

La diferencia entre la DRU y la ERS NO es que la DRU emplee lenguaje natural y la ERS emplee modelos o notaciones formales. Tanto la DRU como la ERS pueden utilizar todo tipo de notaciones. La diferencia entre ambos documentos es más de **nivel de detalle**.

Ejemplo de DRU vs. ERS

Veamos la diferencia en un ejemplo:

- **Definición de requisitos: DRU**

«El software debe acceder a archivos externos creados por otras herramientas»

- **Especificación de requisitos: ERS**

1.1. El usuario debe poder elegir el tipo de archivo externo

1.2. Cada tipo de archivo externo se puede asociar con la herramienta que produce ese tipo de archivo

1.3. Cada tipo de archivo externo puede ser representado por un icono en la barra de herramientas

1.4. Este icono puede ser definido por el usuario

1.5. Cuando el usuario selecciona uno de estos iconos el efecto es aplicar la herramienta asociada a ese tipo de archivo

Puede verse cómo **1 RU: 5 ERS**, un único requisito de usuario da lugar a cinco requisitos software con muchos más detalles.

Validación y gestión requisitos

- Existen varias **técnicas de validación** de requerimientos que pueden usarse en conjunto o de forma individual:

Técnica	Descripción
Revisiones de requerimientos	Los requerimientos son analizados sistemáticamente por un equipo de revisores.
Construcción de Prototipos	Se muestra un modelo ejecutable del sistema a los usuarios finales y a los clientes.
Generación de casos de prueba	Los requerimientos deben poder probarse, si las pruebas son parte del proceso de validación, se pueden revelar problemas en los requerimientos.
Análisis de consistencia automática	Si los requerimientos se expresan como un modelo del sistema en una notación estructurada o formal, para poder hallar inconsistencias.



Ejemplos de herramientas

- SonarQube**: Plataforma de análisis de código estático que puede detectar una amplia gama de problemas en el código.
- Coverity Scan**: Herramienta de análisis de código. estático que puede detectar vulnerabilidades de seguridad en el código.
- CodePro Analytix**: Herramienta de análisis de código estático que puede detectar errores potenciales en el código y medir la complejidad del código
- IntelliJ IDEA**: IDE con muchas funciones de análisis de código estático integradas.
- Eclipse**: IDE con algunas funciones de análisis de código estático integradas.

Especificaciones de los requerimientos

1*Requerimientos Funcionales

- Definen las funciones que el sistema será capaz de realizar.

Son los requisitos relativos a las capacidades y servicios.

Describen las transformaciones que el sistema realiza sobre las entradas para producir salidas.

Describen servicios o funciones

2*Requerimientos No funcionales

- Son restricciones o limitaciones de los servicios o funciones ofrecidos por el sistema.

Son un límite en el sistema o en el proceso de desarrollo.

Incluyen restricciones de tiempo, sobre el proceso de desarrollo, estándares, fiabilidad, respuesta en tiempo, capacidad de almacenamiento, portabilidad, rendimiento, etc.

Son relativos a atributos de calidad del sistema mientras desempeña sus funciones

3*Requerimientos del dominio

- Son requerimientos que provienen del dominio de aplicación del sistema y que reflejan las características de ese dominio.

→Estos pueden ser **funcionales** o **no funcionales**.

4*Restricciones del Diseño

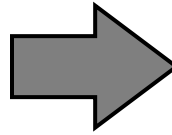
- Son condicionantes existentes para el diseño, sin anticiparlo.

•Aseguran que se cumplan los objetivos del proyecto sin comprometer la calidad y funcionalidad del producto final

También pueden ser

Si hay incumplimiento en los requerimientos...

Funcionales: Lo que el producto **debe hacer** (alcance del sistema, requerimientos Funcionales y de datos)



No funcionales: establecen restricciones de **cómo** estos requisitos funcionales **son implementados**



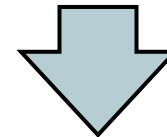
Debe Hacer



Degradará el sistema



Debe Tener

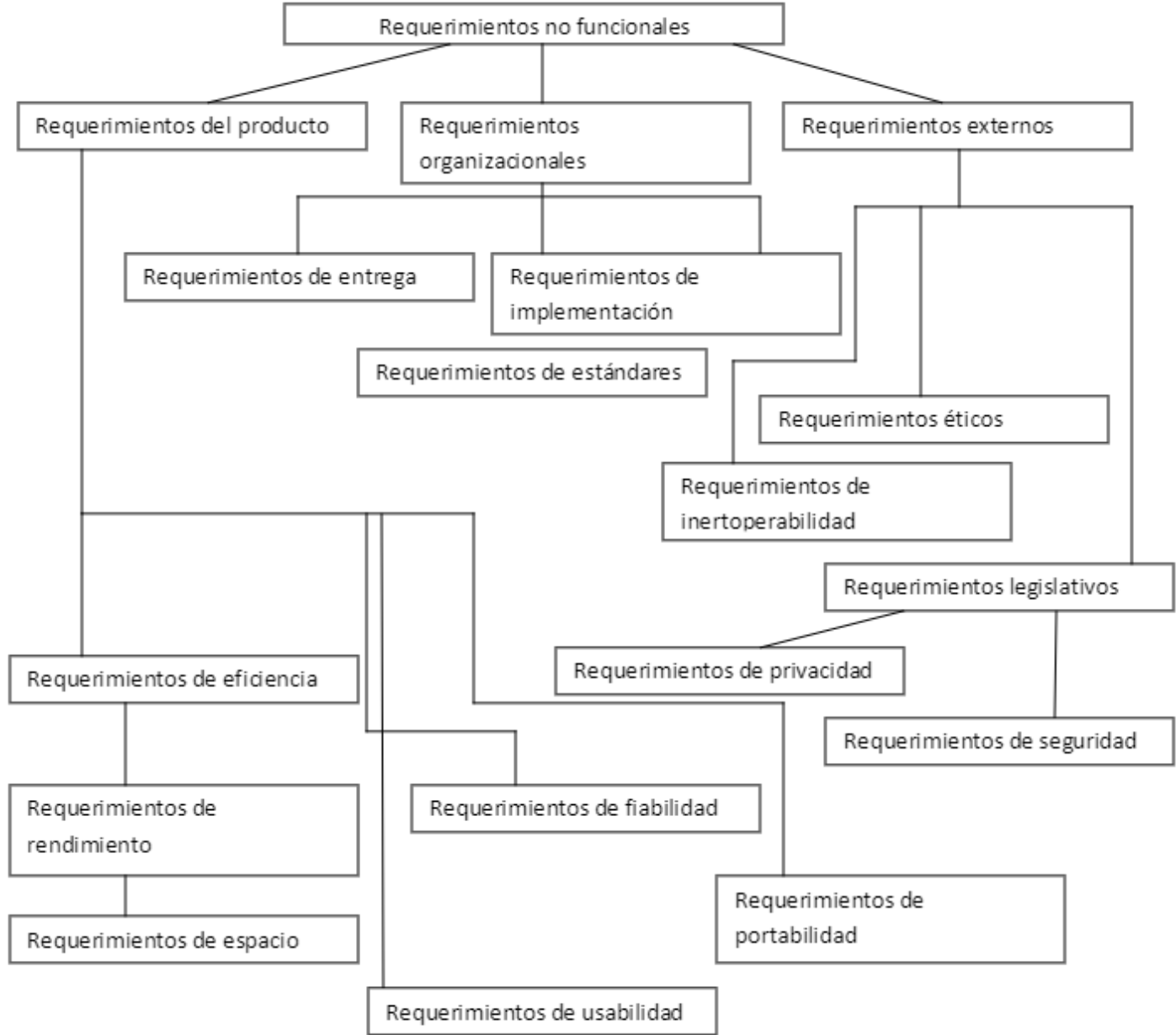


Inutiliza el sistema



A menudo son más críticos que los requerimientos funcionales particulares.

Ejemplo de requerimientos no funcionales



De qué depende que un requerimiento se exprese como funcional o como no funcional..??

..... **depende del grado de detalle** con el que desee incluirse entre los requerimientos el documento de requerimientos

.... o del **grado de confianza** que existe entre el cliente y el equipo desarrollador del sistema

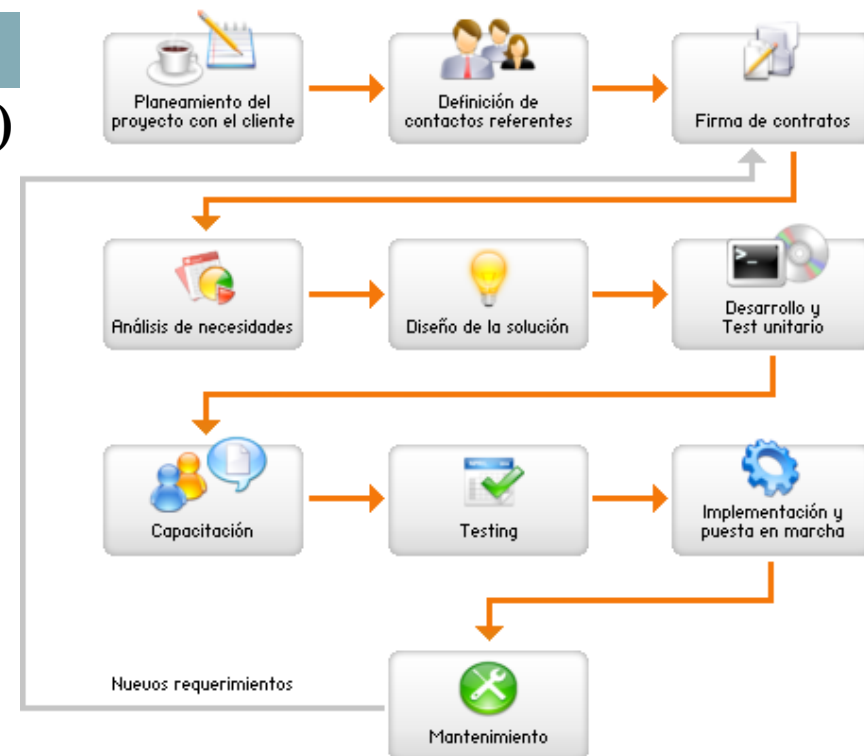
Confidence
is
Contagious

6-Métodos de ingeniería de software

- Una metodología puede seguir uno o varios modelos de ciclo de vida, es decir, el ciclo de vida indica qué es lo que hay que obtener a lo largo del desarrollo del proyecto pero no cómo hacerlo.
- La **metodología** indica **cómo hay que obtener** los distintos productos parciales y finales.

METODOLOGÍAS

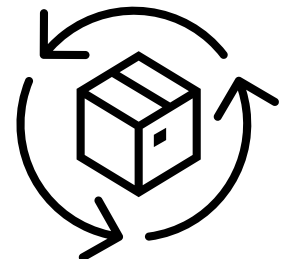
- Desarrollo Convencional (Sin Metodología)
- Desarrollo Estructurado.
 - Orientadas a Procesos
 - Orientadas a datos
 - Jerárquicas
 - No Jerárquicas
 - Mixtas
- Desarrollo Orientado a Objetos.
- Tradicionales y Ágiles



7-Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

Ciclo de Vida

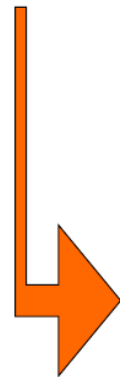
- Es una sucesión de etapas por las que pasa el software en su desarrollo, desde que se concibe la idea hasta que el software deja de utilizarse (obsolescencia)
- “Un marco de referencia que contiene los procesos, las actividades y las tareas involucradas en el desarrollo, la explotación y el mantenimiento de un producto de software, abarcando la vida del sistema desde la definición de los requisitos hasta la finalización de su uso” ISO 12207-1



Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.

Cada **etapa** lleva asociada una serie de actividades y tareas que se deben realizar, y una serie de documentos que serán la salida de cada una de estas fases y que servirán de entrada a la fase siguiente

Ciclo de vida $\neq$ Ciclo de desarrollo

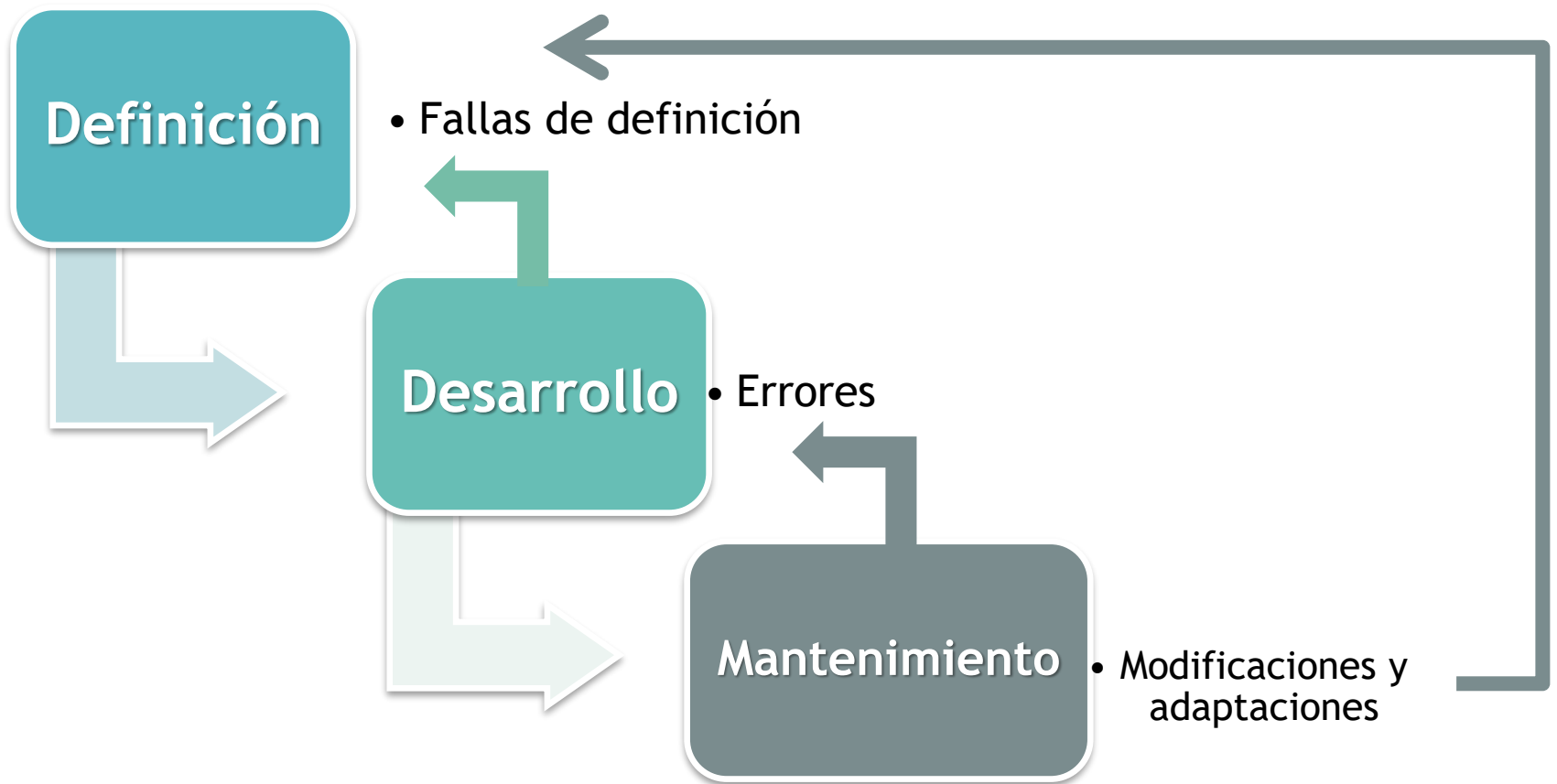


Toda la vida del sistema:
desde la concepción hasta
el fin de uso



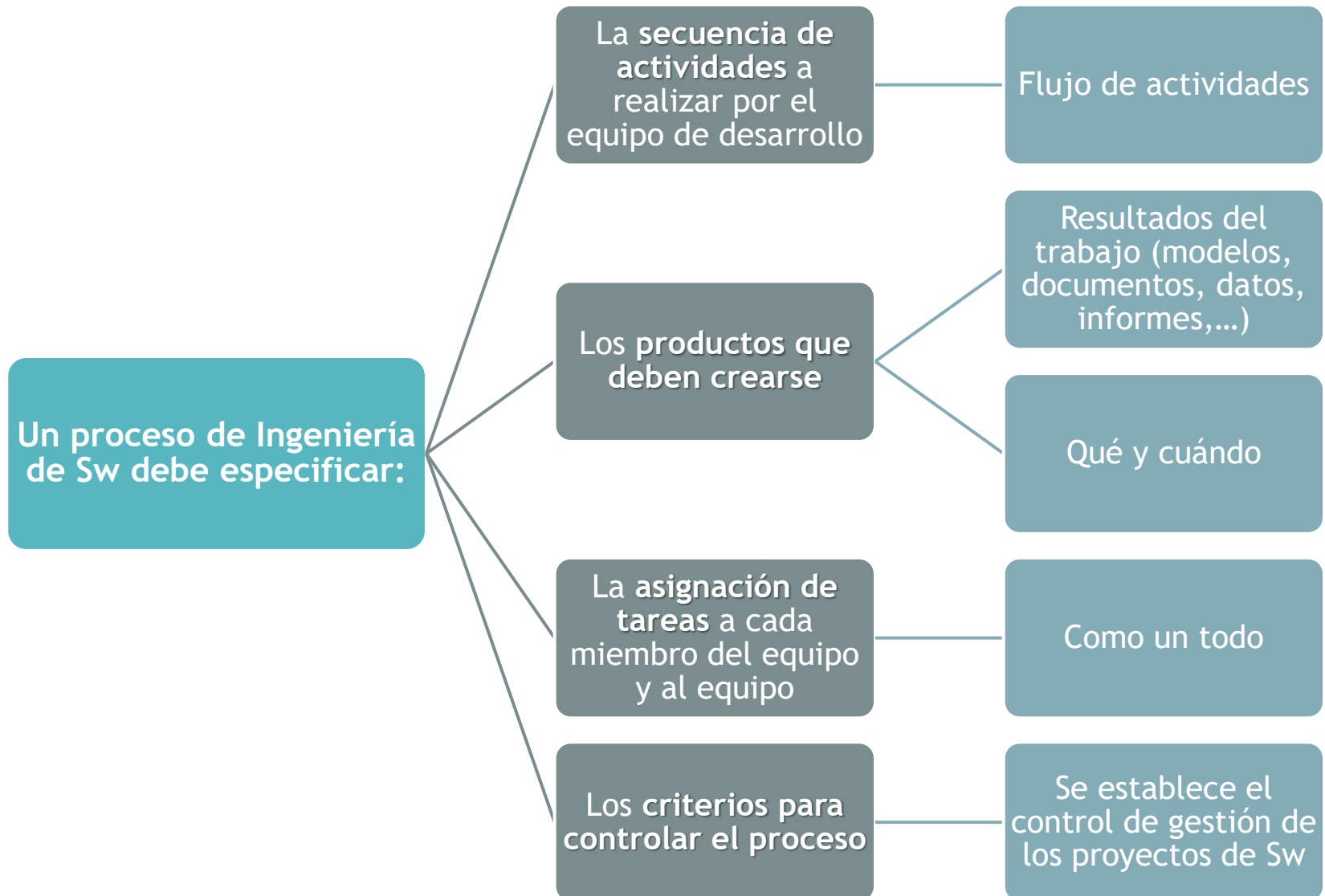
Desde el análisis
hasta la entrega
al usuario

Proceso de ingeniería de software



En la practica, no es secuencial: debe ser **iterativo e incremental** (procesos, no fases)

Proceso de Ingeniería de Sw



Problemas en el Modelo del Proceso

- Normalmente, las especificaciones son incompletas o anómalas
- No existe una distinción precisa entre la especificación, el diseño y la producción.
- Recién cuando el sistema se ha producido se puede probar
- No siempre se puede remplazar el sistema o partes, durante el mantenimiento



Modelo de proceso	Funciona con requisitos y arquitectura no predefinidos	Produce software altamente fiable	Gestión de riesgos	Permite correcciones sobre la marcha	Visión del progreso por el Cliente y el Lider del proyecto
Codificar y corregir	Bajo	Bajo	Bajo	Alto	Medio
Cascada	Bajo	Alto	Bajo	Bajo	Bajo
Evolutivo exploratorio	Medio o Alto	Medio o Alto	Medio	Medio o Alto	Medio o Alto
Evolutivo prototipado	Alto	Medio	Medio	Alto	Alto
Desarrollo formal de sistemas	Bajo	Alto	Bajo a Medio	Bajo	Bajo
Desarrollo orientado a reutilización	Medio	Bajo a Alto	Bajo a Medio	Alto	Alto
Incremental	Bajo	Alto	Medio	Bajo	Bajo
Espiral	Alto	Alto	Alto	Medio	Medio

Que modelo utilizar ?

- Para sistemas bien comprendidos utiliza el **Modelo de Cascada**. La fase de análisis de riesgos es relativamente fácil.
- Con requerimientos estables y sistemas de seguridad críticos, utiliza **modelos formales**.
- Con especificaciones incompletas, utiliza el **modelo de prototipado**.
- Pueden utilizarse **modelos híbridos** en distintas partes del desarrollo.

Características de los Productos de Software

Mantenibles	• Debe ser posible que el software evolucione y que siga cumpliendo con sus especificaciones
Confiabilidad	• El software no debe causar daños físicos o económicos en el caso de fallos.
Eficiencia	• El software no debe desperdiciar los recursos del sistema.
Utilización adecuada	• El software debe contar con una interfaz de usuario adecuada y su documentación.
Entendible	• Se encuentra el proceso bien definido y es entendible
Visible	• El proceso es visible al exterior ?
Soportable	• Puede el proceso ser soportado por herramientas CASE ?
Aceptable	• El proceso es aceptado por aquellos involucrados en el ?
Seguridad	• Los errores del proceso son descubiertos antes de que se conviertan en errores del producto ?
Robusto	• Puede continuar el proceso a pesar de problemas inesperados ?
Escalable	• Puede el proceso evolucionar para cumplir con los objetivos organizacionales ?
Rapido	• Que tan rápido puede producirse el sistema ?

8-Retos de la Ingeniería de Software

De lo Heredado

El reto es “mantener y actualizar” el Sw tal que se eviten los costos excesivos y que los servicios esenciales del negocio sigan funcionando

De la Heterogeneidad

Desarrollar técnicas para “construir un Sw confiable” que sea lo suficientemente flexible.

Heterogéneo: diferentes clases de sistemas, redes y sistemas de soporte

De la entrega

El reto es” reducir los tiempos de entrega” para sistemas grandes y complejos sin comprometer la calidad del sistema

No son independientes: hacer frente a los sistemas heredados (legacy systems), hacer frente a la creciente diversidad y hacer frente a las demandas de entrega con tiempos cada vez más reducidos.

9-Usabilidad en Desarrollo de software



Calidad en interfaces de usuario (A aplicar en TP)

Calidad percibida

- Conjuntos de características de un producto que aportan una gran satisfacción a unos usuarios específicos.

Calidad Externa

- Instante en el cual un producto satisface las necesidades establecidas e implícitas cuando este es utilizado bajo ciertas condiciones específicas.

Calidad en el uso

- La **efectividad** (precisión y completitud de alcanzar objetivos), **eficiencia** (usar los recursos necesarios en relación con la precisión y completitud con la que los usuarios alcanzan los objetivos) y **satisfacción** (aceptación del sistema por parte de los usuarios) con el cual ciertos usuarios pueden alcanzar ciertas metas en entornos concretos

Usabilidad

- Instante en el cual un producto puede ser utilizado por usuarios específicos para alcanzar metas específicas con efectividad, eficiencia y satisfacción en un contexto de uso específico.

Usabilidad

- Es la forma en que se alcanzan los objetivos con efectividad, eficiencia y satisfacción.
- La usabilidad, hace referencia, a la **rapidez y facilidad con que las personas llevan cabo sus tareas propias a través del uso del producto objeto de interés**, idea que descansa en cuatro puntos:

Una aproximación al usuario:

enfocarse en los usuarios.

Se tienen que conocer, entender y trabajar con las personas que representan a los usuarios actuales o potenciales del producto.

Un amplio conocimiento del contexto de uso:

Las personas utilizan los productos para incrementar su propia productividad.

Un producto se considera fácil de aprender y usar en términos del tiempo que toma el usuario para llevar a cabo su objetivo, el número de pasos que tiene que realizar para ello, y el éxito que tiene en predecir la acción apropiada para llevar a cabo.

El producto ha de satisfacer las necesidades del usuario:

Los usuarios son gente ocupada intentando llevar a cabo una tarea.

Se va a relacionar usabilidad con productividad y calidad.

Son los usuarios, y no los diseñadores y los desarrolladores, los que determinan cuando un producto es fácil de usar.

Usabilidad

Que toma en cuenta la Usabilidad?

Rapidez y facilidad de hacer las cosas

Aproximación al usuario

Conocimiento del contexto de uso

Satisfacer las necesidades del usuario

Los usuarios son los que determina si el producto es fácil de usar

Importancia

Una reducción de los costos de producción.

Reducción de los costos de mantenimiento y apoyo.

Reducción de los costos de uso

Mejora en la calidad del producto

Cuándo se considera la usabilidad?

Independientemente del método de desarrollo del sistema, idealmente **debe de considerarse a la usabilidad a lo largo de todo el proyecto**

Para así no tener que hacer cambios drásticos en etapas futuras del sistema

Calidad en interfaces de usuarios

Web Accessibility
Initiative del
W3C

Accesible = usabilidad universal

- Uso equitativo
- Uso flexible, diseñado para personas con distintos gustos y habilidades
- Usos simple e intuitivo, fácil de entender
- Información perceptible, el diseño comunica al usuario la información necesaria
- Tolerancia para el error
- Esfuerzo físico mínimo
- Tamaño y espacio para poder aproximarse y usar el diseño

Calidad en interfaces de usuario

DISCAPACIDAD	PERSONAS DISCAPACITADAS	PERSONAS SIN DISCAPACIDAD
Sin visión	Ciegos 	Con ojos ocupados - En la oscuridad
Poca visión	Con limitaciones visuales 	Manejando un visualizador pequeño
Sin audición	Sordos 	Oídos ocupados - silencio forzado
Oído limitado	Con poca audición 	En entornos ruidosos
Manipulación limitada	Movilidad limitada, falta de extremidades 	Ropa especial - vehículo inestable
Limitaciones cognitivas	Con problemas cognitivos Otras discapacidades	Distraídas - alcoholizadas - sueño - drogados
Lectura limitada		Desconocen la lengua

NO SE OLVIDEN DE LEER

(hay preguntas para los parciales y globales)

**Sommerville:
págs. 327 a 347**

**Pressman,
7ma. Ed: págs.
265 a 292**

10-Introducción a la Guia SWEBOK

El objetivo principal de SWEBOK (Software Engineering Body of Knowledge) es **proporcionar una guía completa y actualizada del cuerpo de conocimiento de la Ingeniería de Software.**



En el proyecto también están representados:

- los dos principales organizaciones de estandarización en Ingeniería del Software: IEEE e ISO/IEC JTC1/SC/.
- Los autores de las tres principales obras de Ingeniería del Software: **Steve Mc Connell, Roger Pressman e Ian Sommerville.**
- Universidad de Québec (Montreal)
- Empresas y organizaciones como: Rational, SAP, Boeing, Construx, MITRE, Raytheon, etc.

SWEBOK se utiliza como referencia por:

- **Profesionales de la Ingeniería de Software** para mejorar sus habilidades y conocimientos.
- **Educadores** para desarrollar planes de estudio y cursos de Ingeniería de Software.
- **Organizaciones** para definir sus procesos de desarrollo de software.
- **Investigadores** para identificar nuevas áreas de investigación en Ingeniería de Software.

10-Guía SWEBOK

La guía SWEBOK persigue tres fines relacionados:

1. **Identificar aquellas partes del cuerpo de conocimientos sobre las cuales existe un consenso generalizado entre los miembros de la comunidad académica y profesional de la ingeniería del *software*.**
2. **Organizar estas partes en áreas de conocimiento.**
3. **Crear los medios necesarios para acceder a ellas.**

Para cumplir con estos objetivos la IEEE CS ofrece esta guía en varios formatos, dos de ellos se distribuyen de manera gratuita y están disponibles en la dirección www.swebok.org.

- ❖ La versión 3.0 de la guía SWEBOK organiza el conocimiento de la ingeniería del *software* en **quince secciones denominadas áreas de conocimiento**:

Requisitos de
software.

Diseño de software.

Construcción de
software.

Pruebas de
software.

Mantenimiento de
software.

Gestión de la
configuración del
software.

Gestión de la
ingeniería del
software.

Procesos de
ingeniería del
software.

Modelos y métodos
de ingeniería del
software.

Calidad del
software.

Práctica
profesional de la
ingeniería del
software.

Economía de la
ingeniería del
software.

Fundamentos de
computación.

Fundamentos
matemáticos.

Fundamentos de
ingeniería.



*Guide to the Software
Engineering Body of Knowledge*

Table I.1. The 15 SWEBOK KAs

Software Requirements
Software Design
Software Construction
Software Testing
Software Maintenance
Software Configuration Management
Software Engineering Management
Software Engineering Process
Software Engineering Models and Methods
Software Quality
Software Engineering Professional Practice
Software Engineering Economics
Computing Foundations
Mathematical Foundations
Engineering Foundations

El cuerpo de conocimiento identificado por el proyecto SWEBOK se ha configurado como el estudio más relevante y como la referencia de más autoridad en toda la comunidad informática para la acotación y descripción de los conocimientos que configuran la Ingeniería del software.

The Guide to the Software Engineering Body of Knowledge (SWEBOK Guide) was established with the following five objectives:

1. To promote a consistent view of software engineering worldwide
2. To specify the scope of, and clarify the place of software engineering with respect to other disciplines such as computer science, project management, computer engineering, and mathematics
3. To characterize the contents of the software engineering discipline
4. To provide a topical access to the Software Engineering Body of Knowledge
5. To provide a foundation for curriculum development and for individual certification and licensing material